

Claude Code for research

Bahlo lab meeting

Stuart Lee

2026-05-12

**Claude Code is
research infrastructure,
not autocomplete.**

Three threads today

1. How it works.
2. How I use it.
3. What to watch for.

Then group activities.

How it works

An LLM chat is one long prompt that grows with every turn

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-06-28  
Reasoning: high  
# Valid channels: analysis, commentary, final.  
Channel must be included for every message.  
Calls to these tools must go to the commentary channel: 'functions'.<|end|>
```

system prompt
(written by openai)

```
<|start|>developer<|message|># Instructions  
Be concise and technical. Provide code examples when relevant.
```

developer message
(tool definitions)

```
# Tools  
## functions  
namespace functions {  
  type web_search = {:  
    query: string,  
    num_results?: number,  
  } => any;  
} // namespace functions<|end|>
```

```
<|start|>user<|message|>What's new in the latest version of polars?<|end|>
```

user prompt

```
<|start|>assistant<|channel|>analysis<|message|>User asking about latest polars version.  
My knowledge cutoff is 2024-06, need to search for recent updates.<|end|>
```

thinking

```
<|start|>assistant<|channel|>commentary<|message|><|call|>functions.web_search  
{  
  "query": "polars python latest version new features",  
  "num_results": 5  
><|end|>
```

tool call

```
<|start|>tool<|message|>[{"title": "Polars 1.8.0 Release Notes", "snippet": "New  
streaming engine, improved join performance, added pivot_wider...", "url": "..."}]<|end|>
```

tool results

```
<|start|>assistant<|channel|>final<|message|>Polars 1.8.0 (latest as of Nov 2025)  
includes:
```

```
- **Streaming engine overhaul**: Better memory efficiency for large datasets  
- **Join performance**: 2-3x faster for certain join types  
- **New API**: 'pivot_wider()' for reshaping data  
- **LazyFrame improvements**: Enhanced query optimization
```

final response

```
Check the release notes for migration details if upgrading from older versions.<|return|>
```

ANATOMY OF A
MODERN LLM
CHAT

You
typically
see only
these
two

A token is a subword piece — about three-quarters of a word, four characters on average. The context window is measured in tokens (Opus 4.7 on the Team plan: 200K).

Confident wrong answers are a feature of the architecture

Without tools, it generates fluent text — not retrieved facts.

Fluent and **wrong** sound the same.

The context window is the model's only memory

Context window (this turn)

- file you opened
- error you pasted
- tool result

= the model's only memory

Not in the buffer:

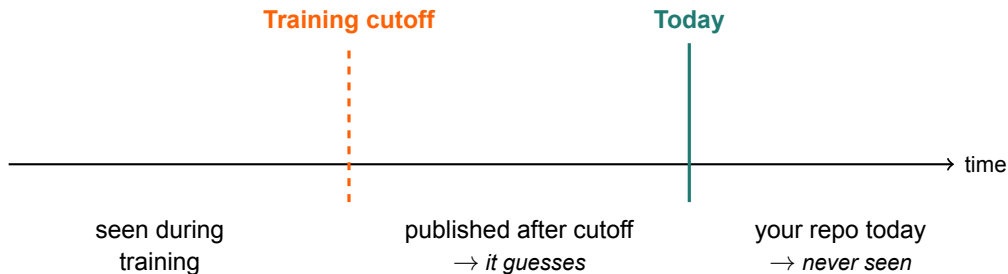
previous session

files you didn't open

tool calls in other turns

If it “forgot” — it never had it.

Training data has a freshness date



Grounding turns guesses into answers we can check

Show the actual file, the actual error, the actual data.

What's in the buffer matters more than **which model** you chose.

From web to the command line

In a web chat, you are the bridge between model and reality

You read code in. You copy answers out. You re-paste errors.

The model has no hands. You are the hands.

Fine for one-off questions. Does not scale to a workflow.

Claude Code, concretely

A coding-agent CLI from Anthropic. Point it at a directory; it gets your files, your shell, git, and the web — plus a loop to use them.

- Runs in your terminal, in VS Code, or at `claude.ai/code`.
- Calls Anthropic's API for each model turn.
- Sees what you point it at.

Models: **Sonnet 4.6** for everyday coding; **Opus 4.7** for harder reasoning and longer context.

Not the only harness — **Codex** (OpenAI), **Gemini CLI**, **OpenCode** do similar work. Lessons here transfer.

An agent reads files, runs commands, loops on results

1. **Think**
2. **Act** — read, run, call.
3. **Observe**
4. Repeat.

Can do unattended

Read files, run shells, edit, web, MCP.

Cannot do

Know your goal unless you say it.

More autonomy = more chances to be **wrong** unnoticed.

MCP servers turn Claude into a research client

MCP (Model Context Protocol) — a shared interface that lets Claude call out to external services and data sources without bespoke glue.

- **PubMed** — search, metadata, full text.
- **bioRxiv / medRxiv** — preprints by topic, author, funder.
- **Microsoft 365** — email, files, calendar.

A lever that makes Claude useful for **research**, not just code.

How I actually use it

**TDD — test-driven development —
gives agents a target you can check**

The test pins Table 1 to the correct case definition

Red — write the invariant first.

```
# fixture: eids 1-2 prevalent, 3-5 incident
cohort <- make_classified_cohort()
tab <- summarise_cohort_characteristics(cohort)

neuro <- tab[tab$variable ==
  "Incident neurological dx (post-baseline)", ]
expect_match(neuro$carrier, "^3 \\(")
```

Green — incident = symptomatic *and* post-baseline.

```
# in format_incident_row()
.incident <- as.integer(
  df[[bin_col]] == 1L &
  !is.na(df[[age_col]]) &
  df[[age_col]] > df$age_at_assessment
)
```

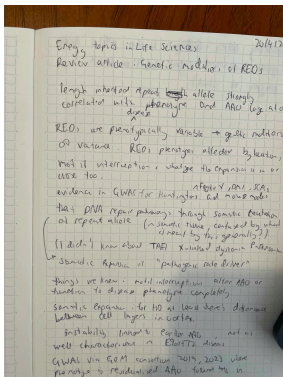
/capture, /meeting, /review make Claude a lab-notebook collaborator

Recurring workflows I wrote once as plain-text files in `.claude/commands/` and invoke with a slash.

- `/capture` — thought, paper, fragment → structured note.
- `/meeting` — meeting transcript → actions, decisions, follow-ups.
- `/review` — week's end → summary, what's stuck, next week.

Not the author. The structure that lets me **write more** of my own thinking down.

A paper read becomes a queryable archive entry



Handwritten while reading.

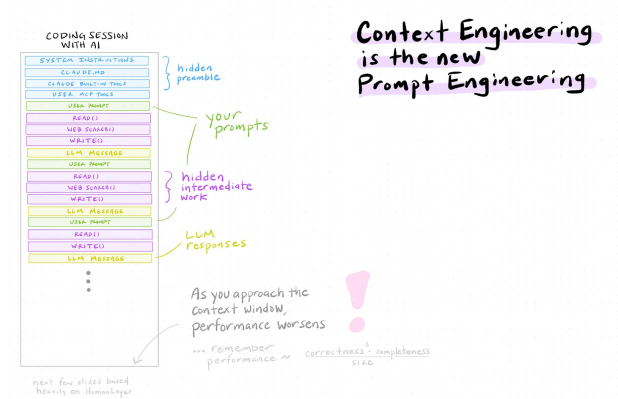
DASHBOARDNOTESENTITIESPROJECTSACTIONS	
Rajagopal, S., Donaldson, J., Flower, M., Hensman Moss, D.J. and Tabrizi, S.J. (2023) 'Genetic modifiers of repeat expansion disorders', <i>Emerging Topics in Life Sciences</i> . Available at: https://doi.org/10.1042/ETLS20230015 .	
#c9orf72 #repeat-expansions #dna-repair #gwas	METADATA [REF] · 2026-04-20
Research Question Review framing: what drives the phenotypic variability of monogenic repeat expansion disorders (REDs) despite shared genotype–phenotype structure? Focus on (i) inherited repeat length, (ii) motif interruptions, and (iii) somatic expansion mediated by DNA damage repair (DDR) pathways as modifying axes.	ENTITIES GWAS (concept) repeat expansions (concept) C9orf72 (project)
Key Points - Length of the inherited repeat allele is the dominant predictor of age-at-onset (AAO) and disease progression across REDs; AAO scales approximately logarithmically with repeat length. - REDs are phenotypically variable despite being monogenic → modifier genetics exist. Phenotype is shaped by the cognate gene, coding vs non-coding location of the repeat, and repeat-sequence interruptions.	RELATED Hujos2026 DNA repeat expansions 900k biobank (3) SCA27B multi-RE burden collaboration (Ponger) (2) What's new in biology spring 2026 (2) Fratta2012-24 (2) Kojak2024-cymouse (2) STR-GWAS and motif info score data paths (2) GEL replication task list (2) RE Working Group (2)

Linked, tagged, searchable from any future session.

One /capture per paper. My notes/ directory is just markdown — Claude reads it the same way I do.

Context management

Context is the bottleneck, not the model



What you put in beats how you phrase it.

CLAUDE.md and skills give Claude durable memory

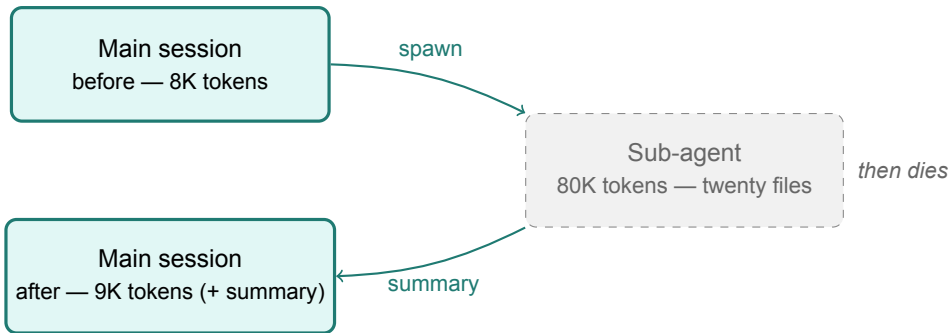
CLAUDE.md — always loaded.

- Conventions
- Domain vocabulary
- Project anti-patterns
- Pointers to source-of-truth files

Skills / slash commands — on demand.

- Reviewing a PR
- Bumping a package version
- Drafting a session log

Sub-agents read so the main session doesn't have to



Design docs survive sessions. **Drift** = repeating mistakes you've already corrected — restart when it starts.

Verification

Git is Claude's other memory — and your safety net

- **Rollback is cheap.** `git reset` undoes nonsense.
- **Memory is auditable.** "What changed in the last three commits?" — concrete, not guessed.

Commit early, commit often.

Read the diff, every time

- Code → run the test, quote the output.
- Data → row counts, dtypes, NA rates, before / after.
- Plot → render it. Look at it.
- Model → predicted vs. actual.

“It should work” is **not** verification.

Simulate with a known truth; check it's recovered

For any step that feeds a figure or a table:

1. Generate data with known ground truth.
2. Run the pipeline.
3. Confirm it recovers the truth within tolerance.

Substitute for cross-software replication. The simulation is the test fixture.

When it's confused, re-ground — don't argue

Vague prompt
“compare the two ancestries”



Plausible-looking wrong
“Northern vs Southern European...”
(silently swapped)

Grounded prompt
“compare gnomAD AFR vs EUR
in /data/.../finemap/”



Verifiable correct
“AFR ($n=8127$) vs EUR
($n=...$)...”

Pitfalls

Six ways this goes wrong

Premature action

plan first

Confabulated paths

verify by reading

Sycophancy

it will agree if you push

Cost runaway

set a budget

Outsourced judgment

uses what you can't verify

Vibe coding

runs, but no one understands it

Working with sensitive data

Data egress is the constraint

Fine

- Code-only HPC repos
- Pipeline + simulation code
- Manuscript drafts (code-only dirs)

Not fine without policy

- UKB, clinical data
- Mixed code/data directories
- "It's on the HPC" as control

It's not *where* the agent runs — it's *what data leaves our networks*.

Local models on the HPC unlock controlled-data work

The path forward

Run **open-weight models** (e.g. the **Qwen** family) on our HPC for work involving controlled-access data — no bytes leave the network.

Until then: laptops ✓, code-only HPC ✓, controlled data **off-limits**.

Anthropic doesn't train on commercial-tier prompts; **Enterprise tier** adds zero-data-retention if we need it.

Where to from here?

Now → next → ongoing

1. **Now** — work through ACTIVITIES.md together.
2. **Next week** — pick one activity, try it on your own work, bring back what broke.
3. **Ongoing** — regular lab catch-up to share tips, skills, and gotchas as we find them.